Proyecto sistemas complejos

Juan Pablo Bustos
Juan Esteban Martinez
David Esteban Martinez

September 2026

1 Definición del Problema

En la actualidad se están implementando redes en las cuales se ejecutan modelos de IA que demandan muchos recursos de procesamiento. Estos recursos no se pueden alojar en un solo servidor dado que el hardware es limitado. Para esto se opta por usar redes de cómputo en las que se pueda asignar parte de ese procesamiento en diferentes servidores, ya que en arquitecturas tradicionales, la computación distribuida depende de servidores o coordinadores centrales (Cloud, clústeres administrados). Esta dependencia centralizada genera tres vulnerabilidades críticas en entornos sin infraestructura fija:

- **Punto Único de Falla:** Si la entidad central colapsa o pierde conectividad, todo el sistema queda inoperativo.
- **Cuellos de Botella:** Si la entidad central, el servidor, todo el sistema queda inoperativo.
- **Incapacidad de Adaptación:** la topología de una red cambia constantemente debido al movimiento de información, provocando a veces desconexiones o interferencias. En sistema centralizado está la limitación de no poder adaptarse a las dinámicas del tráfico y a los cambios locales que estos pueden producir en la red.

Para este proyecto se va diseñar e implmentar un modelo de red descentralizada que simule un procesosamiento de un modelo IA.

2 Requisitos funcionales y no funcionales del sistema

2.1 Requisitos Funcionales

1. El sistema debe descubrir automáticamente nodos activos dentro de la misma red Ad hoc mediante difusión de mensajes.

2. Cada nodo debe publicar su estado de recursos (CPU, Memoria, Ancho de Banda) periódicamente.

3. El planificador debe seleccionar el nodo más apto para ejecutar una tarea según métricas ponderadas.

2.2 Requisitos No Funcionales

1. **Tolerancia a fallos:** La red debe detectar la caída de un nodo en un tiempo máximo de $T_{timeout} = 5$ segundos.

2. **Modularidad:** a API debe desacoplar la capa de comunicación de la lógica de procesamiento mediante interfaces abstraídas.

3 Modelo matemático del sistema

Grafo Dinámico de Red:

$$G(t) = (V(t), E(t))$$

Donde $V(t)$ son los nodos activos y $E(t)$ son los enlaces vigentes en el instante t. Vector de Estado de Nodo:

$$S_i(t) = [C_i(t), M_i(t), B_i(t), R_i(t), A_i(t)]$$

Representa CPU disponible, Memoria libre, Ancho de Banda, Reputación y Disponibilidad. Función de Aptitud para Asignación:

$$F(i, j) = w_1 C_i + w_2 M_i + w_3 B_i + w_4 R_i - w_5 L_i$$

Se asigna la tarea T_j al nodo $i^* = \arg\max_i F(i, j)$ que maximice la puntuación.

4 Arquitectura por Capas y Componentes del Sistema

- **Capa de Aplicaciones Descentralizadas:** Módulos finales desarrollados sobre la plataforma (ej. monitoreo distribuido, procesamiento de datos o modelos de aprendizaje automático).

- **Capa de API de Alto Nivel:** Abstracción para el desarrollador que expone métodos simples (submit(), connect(), broadcast()), ocultando la complejidad del enrutamiento y la distribución de tareas.

- **Capa de Servicios Distribuidos:** Núcleo de la API que gestiona la lógica descentralizada:

  – **Node Discovery:** Administra el descubrimiento dinámico de pares y mantiene la tabla de vecinos activos (PeerTable).

  - **Resource Manager:** Monitorea y publica continuamente el vector de recursos del nodo (C_i, M_i, B_i)
  - **Distributed Scheduler** Evalúa la función de aptitud $F(i, j)$ para determinar el nodo óptimo de ejecución.
  - **Task Manager:** Controla el ciclo de vida de las tareas (creación, despacho, ejecución y retorno de resultados).
  - **Failure Detector & Recovery Manager:** dentifica nodos inactivos por tiempo de espera (timeout) y gestiona la reasignación de tareas fallidas.

- **Capa de Comunicación y Serialización:** Encargada del transporte de datos y del empaquetado/desempaquetado de mensajes en formato estandarizado (JSON).

- **Capa de Infraestructura y Sistema Operativo:** Entorno físico o emulado donde se ejecutan los nodos (Linux, contenedores Docker o máquinas virtuales).

5 Diseño de componentes de la API

La arquitectura de **ComplexNet API** se fundamenta en un conjunto de componentes modulares y desacoplados. Cada uno asume una responsabilidad específica dentro del ciclo de vida de la red distribuida y la gestión de tareas:

- **Node Discovery:** Se encarga de descubrir de forma dinámica la presencia de nuevos nodos en la red Ad hoc, mantener la tabla de vecinos activos (*PeerTable*) y remover los nodos inactivos mediante mecanismos de temporización.

- **Resource Manager:** Monitorea, registra y publica periódicamente la disponibilidad de recursos de hardware local (uso de CPU, memoria RAM disponible, ancho de banda y servicios activos).

- **Message Service:** Gestiona el transporte de datos entre nodos, soportando patrones de comunicación punto a punto (*unicast*), grupo (*multicast*) y difusión amplia (*broadcast*).

- **Distributed Scheduler:** Selecciona de forma autónoma el nodo óptimo para la ejecución de tareas basándose en el cálculo de la función de aptitud, considerando la carga actual, disponibilidad y reputación de los candidatos.

- **Task Manager:** Controla la creación, distribución, ejecución local, seguimiento de estado y recepción de resultados de las tareas enviadas a la red.

- **Synchronization Service:** Sincroniza operaciones concurrentes y mantiene el orden lógico de los eventos entre los nodos cuando el proceso lo requiere.

- **Failure Detector:** Identifica nodos caídos o fuera de rango mediante el rastreo periódico de mensajes de presencia (*beacons* / *heartbeats*) y tiempos de espera de respuesta.
- **Recovery Manager:** Maneja la tolerancia a fallos parciales, reasignando tareas interrumpidas a otros nodos activos y restaurando los estados operativos.
- **Logger and Monitor:** Mantiene un registro centralizado de eventos, métricas de rendimiento, errores y tiempos de ejecución para la trazabilidad de la red.
- **Developer API:** Ofrece una interfaz de programación simplificada e intuitiva para abstraer los servicios distribuidos y permitir la creación de aplicaciones descentralizadas de alto nivel.

5.0.1 Resumen de Responsabilidades

A continuación se sintetiza el propósito de cada componente dentro del sistema:

Componente	**Responsabilidad Principal**
Node Discovery	Descubrir nodos, mantener la tabla de vecinos y purgar inactivos.
Resource Manager	Registrar y publicar recursos de CPU, RAM, red y servicios.
Message Service	Enviar mensajes unicast, multicast o broadcast entre nodos.
Distributed Scheduler	Seleccionar el nodo óptimo considerando carga y capacidad.
Task Manager	Crear, distribuir, ejecutar, monitorear y finalizar tareas.
Synchronization Service	Coordinar operaciones concurrentes y mantener orden lógico.
Failure Detector	Identificar nodos inactivos mediante tiempos de espera.
Recovery Manager	Reasignar tareas y gestionar fallos parciales de la red.
Logger and Monitor	Registrar eventos, errores, tiempos y métricas del sistema.
Developer API	Proporcionar funciones simples para crear aplicaciones P2P.

Table 1: Matriz de componentes de ComplexNet API.

6 Diseño UML

6.1 Diagrama de clases

El diagrama de clases tiene el nodo con su vector de estados, los nodos estan listados en Tabla pares, PlanificadorDistribuido consulta la TablaPares para seleccionar el mejor nodo para el trabajo, el nodo tiene una relacion de composición con vector estado , gestorRecursos y TablaPares, Tabla pares tiene agregacion con nodo. Anexo.

6.2 Diagrama de secuencia

El diagramade secuencia crea la tarea, busca en la lista de nodos candidatos, selecciona el nodo optimo, se le envia la tarea, si el nodo responde a tiempo el nodo ejecuta y devuelve el resultado de la tarea, si no responde en 5 segundos se detecta el fallo y repite el proceso con un nuevo nodo. Anexo.

7 Protocolo de Descubrimiento de Nodos

El protocolo de descubrimiento de nodos de **ComplexNet API** tiene como objetivo permitir que los nodos que ingresan a la red puedan detectar de forma automática a otros nodos activos, sin depender de un servidor o coordinador central.

Para este propósito se utilizará un mecanismo de descubrimiento basado en difusión UDP (*UDP broadcast*). Cada nodo anunciará periódicamente su presencia mediante mensajes de tipo `RESOURCE_BEACON`, definidos previamente en el formato de mensajes del sistema.

7.1 Identificación de los nodos

Cada nodo contará con un identificador único denominado `node_id`. Además, durante el proceso de descubrimiento se almacenará la información necesaria para establecer comunicación con los demás nodos.

Como mínimo, para cada nodo descubierto se registrarán los siguientes datos:

- `node_id`: identificador único del nodo.
- `ip_address`: dirección IP del nodo dentro de la red.
- `port`: puerto utilizado para la comunicación.
- `last_seen`: instante en el que se recibió el último mensaje del nodo.
- `status`: estado actual del nodo, que puede ser `ONLINE` u `OFFLINE`.

Cada nodo mantendrá localmente esta información en una estructura denominada *PeerTable*. Al mantener una tabla de vecinos independiente en cada nodo, el sistema evita depender de un registro centralizado de participantes.

7.2 Mecanismo de descubrimiento

Cuando un nodo inicia su ejecución, comienza inmediatamente a transmitir mensajes `RESOURCE_BEACON` mediante UDP broadcast.

El mensaje permite anunciar simultáneamente la presencia del nodo y el estado actual de algunos de sus recursos, incluyendo dirección IP, puerto, CPU disponible, memoria RAM disponible y número de tareas activas.

Cuando un nodo recibe un `RESOURCE_BEACON`, ejecuta el siguiente procedimiento:

1. Obtiene el identificador del nodo emisor mediante el campo `sender_id`.
2. Verifica si el nodo ya se encuentra registrado en su *PeerTable*.
3. Si el nodo no existe, crea un nuevo registro y lo marca como `ONLINE`.
4. Si el nodo ya existe, actualiza su información.
5. Actualiza el valor de `last_seen` con el tiempo de recepción del mensaje.

De esta manera, un nodo puede incorporarse dinámicamente a la red sin conocer previamente las direcciones IP de los demás participantes.

7.3 Anuncio periódico de presencia

Los mensajes `RESOURCE_BEACON` también funcionan como mecanismo de verificación de disponibilidad.

Se propone que cada nodo transmita un mensaje de presencia cada 2 segundos. Cada mensaje recibido actualiza el campo `last_seen` correspondiente al nodo emisor.

El comportamiento puede representarse de forma simplificada de la siguiente manera:

Listing 1: Intercambio periódico de mensajes de presencia

```
Nodo A                                  Nodo B

RESOURCE_BEACON  ------------------>
                 <------------------  RESOURCE_BEACON

RESOURCE_BEACON  ------------------>
                 <------------------  RESOURCE_BEACON
```

Este intercambio permite que cada nodo mantenga una visión local y actualizada de los vecinos disponibles en la red.

7.4 Detección de nodos inactivos

Cada nodo verificará periódicamente el tiempo transcurrido desde el último mensaje recibido de cada vecino.

De acuerdo con los requisitos no funcionales definidos para el sistema, un nodo se considerará inactivo cuando hayan transcurrido más de 5 segundos sin recibir un nuevo `RESOURCE_BEACON`.

La condición de detección puede expresarse como:

$$T_{actual} - T_{last_seen} > T_{timeout}$$

donde:

$$T_{timeout} = 5 \text{ segundos}$$

Cuando se cumple esta condición, el nodo correspondiente cambia su estado de:

$$\texttt{ONLINE} \rightarrow \texttt{OFFLINE}$$

Los nodos marcados como `OFFLINE` dejan de ser considerados candidatos activos para la asignación de nuevas tareas.

7.5 Reincorporación de un nodo

Si un nodo previamente marcado como `OFFLINE` vuelve a conectarse a la red, comenzará nuevamente a emitir mensajes `RESOURCE_BEACON`.

Cuando los demás nodos reciban uno de estos mensajes, reconocerán su `sender_id`, actualizarán su dirección IP, puerto, recursos y tiempo de último contacto, y modificarán nuevamente su estado a `ONLINE`.

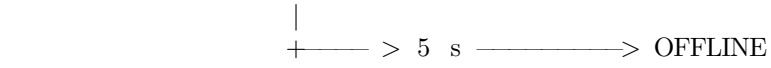
Este comportamiento permite que la topología de la red cambie dinámicamente durante la ejecución del sistema.

7.6 Flujo general del protocolo

El funcionamiento general del protocolo puede resumirse mediante la siguiente secuencia:

Listing 2: Flujo general del protocolo de descubrimiento

```
Inicio del nodo
      |
      v
Enviar RESOURCE_BEACON por UDP broadcast
      |
      v
Escuchar RESOURCE_BEACON de otros nodos
      |
      v
Consultar PeerTable
      |
      +---- Nodo nuevo ----> Registrar nodo
      |
      +---- Nodo conocido -> Actualizar nodo
      |
      v
Actualizar last_seen
      |
      v
Verificar timeout
      |
      +---- <= 5 s ---------> ONLINE
```



7.7 Algoritmo conceptual

El comportamiento del servicio de descubrimiento puede representarse mediante el siguiente pseudocódigo:

Listing 3: Pseudocódigo del protocolo de descubrimiento

```
INICIAR_NODO()

    iniciar_escucha_udp()

    mientras nodo_activo:

        cada 2 segundos:
            enviar RESOURCE_BEACON por broadcast

        si se recibe RESOURCE_BEACON:

            obtener sender_id

            si sender_id no existe en PeerTable:
                registrar nuevo nodo

            actualizar informacion del nodo
            actualizar last_seen
            marcar nodo como ONLINE

        para cada nodo en PeerTable:

            si tiempo_actual - last_seen > 5 segundos:
                marcar nodo como OFFLINE
```

Este protocolo permite realizar descubrimiento dinámico, detección básica de fallos y reincorporación de nodos utilizando únicamente interacciones locales entre los participantes de la red. De esta manera, el sistema conserva su naturaleza descentralizada y puede adaptarse a cambios en la topología sin depender de una entidad coordinadora central.

8 Formato de mensajes

8.1 Formato de Mensajes (Estructura de Datos)

Para garantizar la interoperabilidad entre los nodos la comunicación se realizará mediante mensajes en formato JSON. Todos los mensajes se dividen en una

cabecera (*header*) y un contenido específico (*payload*).

8.1.1 Estructura General del Mensaje

La cabecera contiene la información requerida para el procesamiento y filtrado de mensajes en la red:

- `message_id`: Identificador único del mensaje (UUID v4) para evitar procesamiento duplicado.
- `sender_id`: Identificador o nombre único del nodo emisor.
- `type`: Tipo de operación requerida (`RESOURCE_BEACON`, `TASK_SUBMIT`, `TASK_RESULT`).
- `timestamp`: Marca de tiempo de emisión del mensaje en milisegundos.

8.1.2 1. Anuncio de Presencia y Recursos (RESOURCE_BEACON)

Mensaje periódico emitido por cada nodo mediante difusión UDP para notificar su presencia y el estado actual de sus recursos al resto de la red.

Listing 4: Mensaje JSON para anuncio de recursos

```
{
  "header": {
    "message_id": "msg-001-beacon",
    "sender_id": "nodo-01",
    "type": "RESOURCE_BEACON",
    "timestamp": 1772886922
  },
  "payload": {
    "ip_address": "192.168.1.15",
    "port": 9999,
    "cpu_free_pct": 85.5,
    "ram_free_mb": 2048,
    "active_tasks": 1
  }
}
```

8.1.3 2. Asignación de Tareas (TASK_SUBMIT)

Mensaje dirigido enviada por el planificador (*Distributed Scheduler*) hacia un nodo específico para solicitar la ejecución de un trabajo.

Listing 5: Mensaje JSON para envío de tareas

```
{
  "header": {
    "message_id": "msg-002-task",
    "sender_id": "nodo-01",
```

```
    "type": "TASK_SUBMIT",
    "timestamp": 1772886930
  },
  "payload": {
    "task_id": "task-regresion-01",
    "task_name": "regresion_lineal",
    "parameters": {
      "dataset": "dataset.csv",
      "learning_rate": 0.01
    }
  }
}
```

8.1.4 3. Retorno de Resultados (TASK_RESULT)

Mensaje de respuesta enviado por el nodo trabajador de regreso al nodo emisor al completar el procesamiento de una tarea.

Listing 6: Mensaje JSON para entrega de resultados

```
{
  "header": {
    "message_id": "msg-003-result",
    "sender_id": "nodo-02",
    "type": "TASK_RESULT",
    "timestamp": 1772886945
  },
  "payload": {
    "task_id": "task-regresion-01",
    "status": "COMPLETED",
    "execution_time_sec": 1.45,
    "result_data": {
      "slope": 2.35,
      "intercept": 0.81
    }
  }
}
```

9 Modelo de asignación de tareas

9.1 Objetivo

Formalizar cómo, dado un grafo dinámico $G(t) = (V(t), E(t))$ y una tarea T_j entrante, el sistema decide a qué nodo $i^* \in V(t)$ despachar esa tarea, usando únicamente información local disponible en la red.

9.2 Formalización del problema

Sea $V(t)$ el conjunto de nodos activos en el instante t (obtenido de TablaPares). Cada nodo $i \in V(t)$ tiene asociado su vector de estado:

$$S_i(t) = [C_i(t), M_i(t), B_i(t), R_i(t), A_i(t)] \tag{1}$$

Antes de evaluar la aptitud, se filtra un conjunto de candidatos aptos:

$$V_{\text{cand}}(t) = \{i \in V(t) \mid A_i(t) = \text{verdadero} \wedge C_i(t) \geq C_{\text{req}}(T_j) \wedge M_i(t) \geq M_{\text{req}}(T_j)\} \tag{2}$$

Esto hace que solo se consideren nodos disponibles que cumplen los requisitos mínimos de la tarea T_j.

9.3 Normalización de las métricas

Como C_i, M_i, B_i y L_i se miden en unidades distintas, se normaliza cada métrica al rango $[0, 1]$ usando min-max sobre los candidatos actuales:

$$X'_i = \frac{X_i - X_{\min}}{X_{\max} - X_{\min}} \tag{3}$$

donde $X \in \{C, M, B\}$ se normaliza de forma directa y L (latencia) se normaliza de forma inversa:

$$L'_i = \frac{L_{\max} - L_i}{L_{\max} - L_{\min}} \tag{4}$$

R_i (reputación) ya se puede definir de origen en el rango $[0, 1]$, por lo que no requiere normalización adicional.

9.4 Función de aptitud

Se toman los criterios con más peso la C_i (CPU disponible) y M_i (memoria), debido a que la carga de trabajo son modelos de IA, un nodo con mucha CPU y memoria libres puede ejecutar la tarea aunque tenga algo más de latencia o menor reputación acumulada.

Por tanto en el esquema de pesos C_i y M_i concentran el 80% del peso total:

Table 2: Pesos asignados a los criterios de selección

Peso	**Criterio**	**Valor**
w_1	CPU disponible	0.40
w_2	Memoria libre	0.40
w_3	Ancho de banda	0.10
w_4	Reputación	0.05
w_5	Penalización por latencia	0.05

Reescribiendo la función de la sección 3 con las métricas normalizadas:

$$F(i, j) = w_1 \cdot C'_i + w_2 \cdot M'_i + w_3 \cdot B'_i + w_4 \cdot R_i - w_5 \cdot (1 - L'_i) \tag{5}$$

con la restricción:

$$\sum_{k=1}^{5} w_k = 1 \tag{6}$$

El nodo asignado es:

$$i^* = \arg \max_{i \in V_{\text{cand}}(t)} F(i, j) \tag{7}$$

9.5 Algoritmo de selección

Algoritmo en pseudocódigo, método `seleccionarNodo(tarea)` de la clase `PlanificadorDistribuido`:

Algorithm 1: Algoritmo de selección de nodos

Función `seleccionarNodo(`T_j, $V(t)$`)`
- $V_{\text{cand}} \leftarrow$ filtrar$(V(t), i \rightarrow i.\text{disponibilidad} == \text{verdadero} \wedge i.\text{cpu} \geq T_j.\text{cpuRequerida} \wedge i.\text{memoria} \geq T_j.\text{memRequerida})$;
- **if** V_{cand} *es vacío* **then**
  - **return null** ; `// Se encola` T_j `y se reintenta en el siguiente ciclo`
- normalizar(V_{cand}) ; `// min-max sobre C, M, B, L`
- mejor $\leftarrow$ **null**;
- mejorScore $\leftarrow -\infty$;
- **foreach** *nodo* $i \in V_{cand}$ **do**
  - score $\leftarrow w_1 \cdot C'_i + w_2 \cdot M'_i + w_3 \cdot B'_i + w_4 \cdot R_i - w_5 \cdot (1 - L'_i)$;
  - **if** *score* $>$ *mejorScore* **then**
    - mejor $\leftarrow i$;
    - mejorScore $\leftarrow$ score;
  - **else if** *score* $==$ *mejorScore* **then**
    - mejor $\leftarrow$ `desempatar(`*mejor, i*`)`;
- **return** mejor;

Función `desempatar(`a, b`)`
- **if** $a.\text{latencia} \neq b.\text{latencia}$ **then**
  - **return** el de menor latencia;
- **return** el de mayor reputación;

9.6 Complejidad

Para una tarea T_j con $|V(t)| = n$ vecinos conocidos, el filtrado es $O(n)$ y la evaluación de $F(i, j)$ sobre los candidatos es $O(n)$ por lo tanto la selección completa es $O(n)$ por tarea.

9.7 Ejemplo numérico

Tabla con estados de nodos de ejemplo:

Table 3: Estados de nodos de ejemplo

Nodo	CPU disponible	Memoria	Ancho de banda	Reputación	Latencia (ms)
N_1	80%	4 GB	50 Mbps	0.9	20
N_2	60%	8 GB	100 Mbps	0.7	10
N_3	90%	2 GB	30 Mbps	0.8	40

Usando las métricas normalizadas de antes (C', M', B', R, L'):

- **Nodo** N_1 $(C' = 0.67, M' = 0.33, B' = 0.29, R = 0.9, L' = 0.67)$:

$$F = 0.4(0.67) + 0.4(0.33) + 0.1(0.29) + 0.05(0.9) - 0.05(0.33) \approx 0.46$$

- **Nodo** N_2 $(C' = 0, M' = 1, B' = 1, R = 0.7, L' = 1)$:

$$F = 0 + 0.4(1) + 0.1(1) + 0.05(0.7) - 0 \approx 0.54$$

- **Nodo** N_3 $(C' = 1, M' = 0, B' = 0, R = 0.8, L' = 0)$:

$$F = 0.4(1) + 0 + 0 + 0.05(0.8) - 0.05(1) \approx 0.39$$

En este ejemplo $i^* = N_2$, la razón principal vista para la selección de este nodo es porque tiene la memoria máxima entre los tres candidatos, el N_1 a pesar de tener buena CPU y memoria intermedias no alcanza a N_2.

10 Prueba de concepto inicial

10.1 Objetivo

Con el propósito de validar de forma práctica el diseño propuesto para **ComplexNet API**, se implementó una prueba de concepto inicial de comunicación entre dos nodos.

El objetivo de esta prueba fue comprobar que dos nodos independientes pueden intercambiar mensajes, descubrirse mutuamente, mantener una tabla local de vecinos y detectar cambios en la disponibilidad de los participantes.

La implementación se realizó en Python utilizando sockets UDP y mensajes serializados en formato JSON, siguiendo la estructura del mensaje `RESOURCE_BEACON` definida anteriormente.

10.2 Implementación

Cada nodo se ejecuta como un proceso independiente y mantiene su propia instancia de la estructura *PeerTable*. Los nodos generan periódicamente mensajes `RESOURCE_BEACON` que contienen información de identificación, dirección, puerto y recursos disponibles.

La implementación utiliza los siguientes parámetros:

- Intervalo de envío de `RESOURCE_BEACON`: 2 segundos.
- Tiempo máximo sin recibir mensajes antes de considerar un nodo inactivo: 5 segundos.
- Formato de serialización: JSON.
- Protocolo de transporte: UDP.
- Identificación de nodos mediante un `node_id` único.

El código fue desarrollado de manera que el mismo programa pueda representar diferentes nodos utilizando parámetros de ejecución. De esta forma, no es necesario disponer de una implementación diferente para cada participante de la red.

10.3 Escenario de validación

Para validar la prueba de concepto se ejecutaron dos nodos lógicos como procesos independientes sobre el mismo computador.

Se utilizó la interfaz de red local `localhost` y dos puertos UDP diferentes:

- `nodo-01`: puerto UDP 9999.
- `nodo-02`: puerto UDP 10000.

Esta modalidad permite probar de forma controlada el intercambio de mensajes entre dos nodos independientes sin requerir dos computadores físicos.

Los nodos fueron ejecutados mediante los siguientes comandos:

Listing 7: Ejecución del nodo 01

```
python src/proof_of_concept/node.py --id nodo-01 --port 9999 --peer-port 10000 -
```

Listing 8: Ejecución del nodo 02

```
python src/proof_of_concept/node.py --id nodo-02 --port 10000 --peer-port 9999 -
```

10.4 Prueba de descubrimiento y comunicación

Al iniciar ambos procesos, cada nodo comenzó a transmitir periódicamente mensajes `RESOURCE_BEACON`.

Cuando un nodo recibió por primera vez un mensaje procedente del otro, registró automáticamente al emisor dentro de su *PeerTable* y lo marcó con estado `ONLINE`.

Como resultado, el `nodo-01` descubrió al `nodo-02` y, de manera independiente, el `nodo-02` descubrió al `nodo-01`.

La tabla local de cada nodo almacenó información como el identificador, dirección IP, puerto, CPU disponible, memoria RAM disponible, tiempo del último mensaje recibido y estado actual.

Este resultado permitió comprobar comunicación bidireccional y descubrimiento entre los dos procesos.

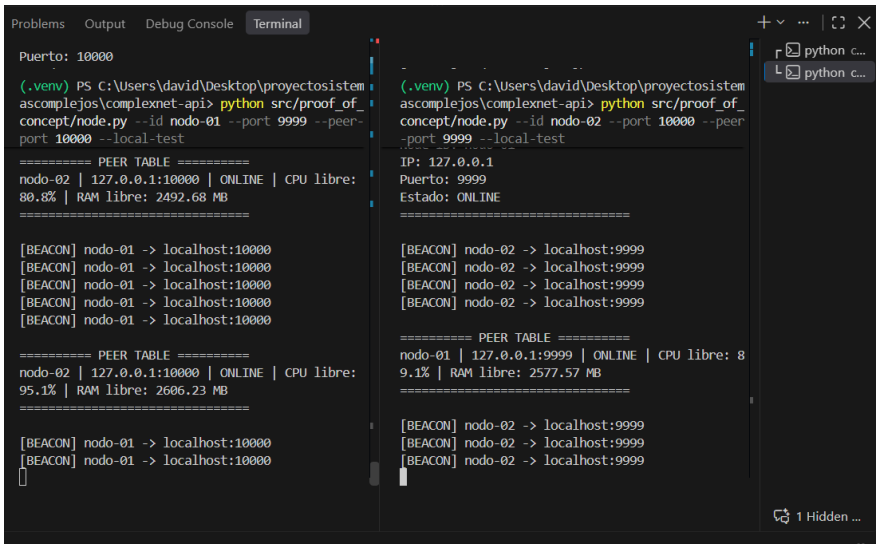


Figure 1: Descubrimiento bidireccional entre `nodo-01` y `nodo-02`. Cada nodo mantiene al otro con estado `ONLINE` en su *PeerTable*.

10.5 Prueba de detección de fallos

Para comprobar el mecanismo de detección de nodos inactivos, se detuvo manualmente la ejecución del `nodo-02` mientras el `nodo-01` continuó funcionando.

Al dejar de recibir mensajes `RESOURCE_BEACON`, el `nodo-01` continuó supervisando el valor `last_seen` asociado al nodo remoto.

Después de superar el tiempo máximo definido de 5 segundos, el sistema detectó la ausencia del nodo y modificó su estado:

$$\texttt{ONLINE} \rightarrow \texttt{OFFLINE}$$

La consola mostró el siguiente evento:

Listing 9: Detección de nodo inactivo

```
[FAILURE] nodo-02 no responde desde hace mas de 5 segundos. Estado: OFFLINE
```

La *PeerTable* también fue actualizada para reflejar el nuevo estado del nodo.

10.6 Prueba de reincorporación

Posteriormente se inició nuevamente el proceso correspondiente al `nodo-02`.

Al comenzar a recibir nuevamente mensajes `RESOURCE_BEACON`, el `nodo-01` identificó que se trataba de un nodo previamente conocido y actualizó su registro.

El estado cambió nuevamente de:

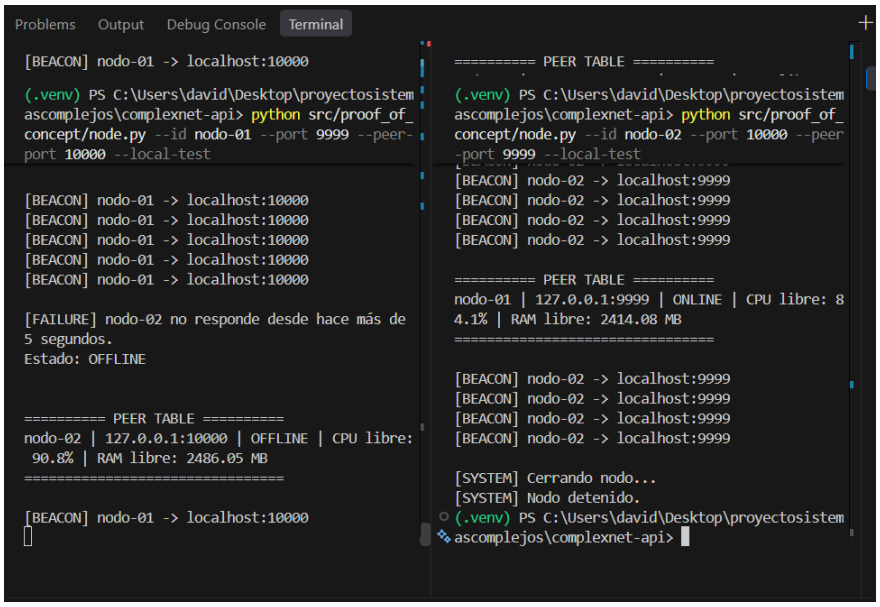


Figure 2: Detección de la caída de `nodo-02`. Después de superar el timeout de 5 segundos, el nodo es marcado como `OFFLINE`.

$$\texttt{OFFLINE} \rightarrow \texttt{ONLINE}$$

La consola registró el evento:

Listing 10: Reincorporación de un nodo

```
[RECOVERY] nodo-02 volvio a estar ONLINE.
```

Esto demuestra que el sistema puede reconocer automáticamente la reincorporación de participantes después de una desconexión.

10.7 Resultados

La prueba de concepto permitió verificar correctamente los siguientes comportamientos:

1. Comunicación entre dos nodos ejecutados como procesos independientes.
2. Descubrimiento bidireccional entre `nodo-01` y `nodo-02`.
3. Intercambio periódico de mensajes `RESOURCE_BEACON`.
4. Creación y actualización de una *PeerTable* local en cada nodo.
5. Publicación de información básica de CPU y memoria RAM.

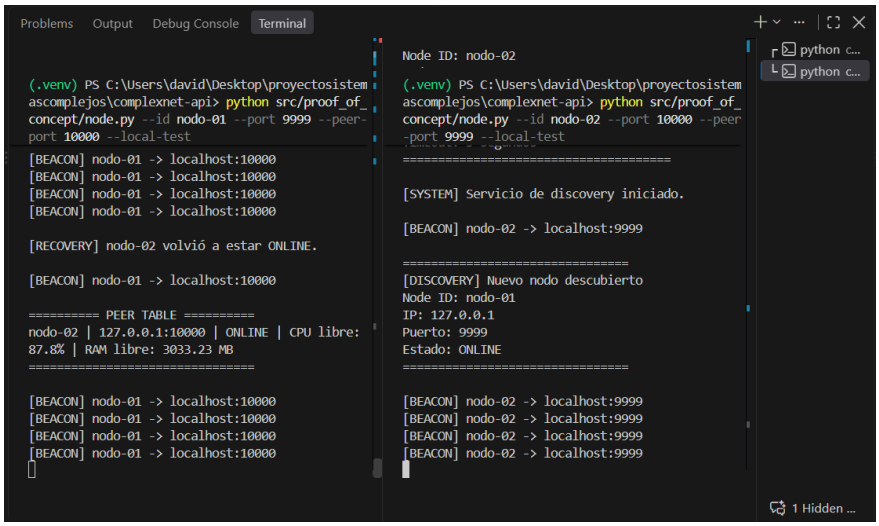


Figure 3: Reincorporación de `nodo-02`. Al volver a recibir mensajes `RESOURCE_BEACON`, su estado cambia nuevamente a `ONLINE`.

6. Detección de un nodo inactivo después de superar el timeout de 5 segundos.
7. Cambio de estado de `ONLINE` a `OFFLINE`.
8. Reincorporación automática de un nodo previamente desconectado.

Los resultados obtenidos permiten validar de forma inicial el mecanismo de comunicación y descubrimiento diseñado para **ComplexNet API**. La prueba constituye una implementación mínima sobre la cual podrán integrarse posteriormente los demás servicios distribuidos del sistema.

11 Descripción del Proyecto – Fase 2

11.1 ComplexNet API: núcleo funcional del sistema

Para la segunda fase del proyecto se construyó el **corazón funcional de ComplexNet API**, es decir, la parte del sistema que permite que varios computadores (llamados *nodos*) trabajen juntos como si fueran un solo equipo, sin depender de un servidor central. Cada nodo es un programa en Python que se conecta a la red, se presenta ante los demás, informa qué recursos tiene disponibles, **alberga un pequeño modelo de inteligencia artificial**, recibe datos y tareas, y devuelve los resultados.

La idea central es sencilla: **si un computador no puede hacer una tarea, otro la hace por él**, y si uno se desconecta, el sistema se reorganiza

solo para que el trabajo no se pierda. Esto es lo que en el curso se denomina un sistema descentralizado, autoorganizado y tolerante a fallos.

11.2 ¿Qué se implementó?

El núcleo funcional quedó conformado por los siguientes servicios, todos exigidos en el alcance mínimo de la asignatura:

Servicio	**Qué hace, en palabras sencillas**
Descubrimiento de nodos	Cada nodo envía periódicamente un mensaje de "estoy aquí y estos son mis recursos". Así los demás lo conocen sin configurarlo a mano.
Registro de recursos	Cada nodo mide su CPU libre, su memoria disponible y declara su ancho de banda. Esa información viaja en el mensaje anterior.
Mensajería	Los nodos se comunican por la red con mensajes en formato de texto (JSON). Pueden hablar con un nodo específico o con todos a la vez.
Planificador distribuido	Cuando llega una tarea, el sistema calcula cuál nodo es el más adecuado para ejecutarla, según sus recursos, su reputación y qué tan rápido responde.
Gestión de tareas	Una tarea se crea, se envía, se confirma, se ejecuta y se devuelve el resultado. Si algo sale mal, se reintenta en otro nodo.
Detección de fallos	Si un nodo deja de dar señales durante unos segundos, el sistema lo marca como *inactivo*.
Recuperación	Las tareas que estaban en el nodo caído se reasignan automáticamente a otro nodo disponible.
Sincronización	Se usa un reloj lógico (reloj de Lamport) para ordenar los eventos entre nodos y saber qué pasó antes que qué.
Registro y monitoreo	Cada nodo guarda un historial y expone una página web sencilla para ver el estado de la red en tiempo real.

Table 4: Servicios implementados en la Fase 2.

11.3 Modelo de Inteligencia Artificial distribuido en los nodos

Un aspecto clave de esta fase es que **la inteligencia artificial no vive en un solo servidor central, sino que está repartida**. Cada nodo de la red contiene su propio modelo de IA (en este caso, un modelo de regresión lineal).

El funcionamiento es el siguiente:

- Un nodo puede generar o recolectar datos (por ejemplo, datos de prueba o mediciones de sensores).

- Ese nodo envía los datos a otro nodo de la red que necesite entrenar o ejecutar su modelo.
- El nodo que recibe los datos utiliza su pequeño modelo de IA para procesarlos (entrenar o predecir) y devuelve el resultado.
- De esta forma, los nodos actúan como pequeños "cerebros" que colaboran entre sí, compartiendo información para resolver problemas más grandes sin necesidad de enviar todo a la nube.

11.4 ¿Cómo funciona, en la práctica?

1. Se encienden **tres nodos** (tres terminales, contenedores Docker o computadores).
2. Cada uno empieza a "saludar" a los demás con un mensaje de presencia.
3. En pocos segundos, los tres se reconocen y construyen una **tabla de vecinos** con la información de cada uno.
4. Desde cualquiera de ellos se envía una tarea y, si es necesario, **datos para el modelo de IA** (por ejemplo, datos para una regresión lineal).
5. El nodo que recibe la tarea calcula cuál vecino tiene más recursos libres y mejor reputación, y se la delega. El nodo elegido usa su modelo de IA para procesar los datos.
6. El nodo elegido ejecuta la tarea, responde con el resultado y todos actualizan sus métricas.
7. Si en medio del proceso **un nodo se apaga**, el resto lo detecta en segundos y **reasigna la tarea** a otro nodo activo.

11.5 Relación con los objetivos del curso

- **Modelo de red dinámico:** el sistema construye en tiempo real un grafo de nodos activos, tal como se plantea en el modelo $G(t) = (V(t), E(t))$.
- **Modelo de asignación:** se implementó la función de aptitud $F(i, j)$ descrita en el documento del proyecto, con pesos configurables.
- **Arquitectura por capas:** la comunicación, el descubrimiento y las tareas están separados en módulos independientes.
- **Aplicación de inteligencia artificial:** se incluyó un modelo de **regresión lineal** en cada nodo, el cual puede entrenarse o predecir usando los datos que le envían otros nodos.

11.6 ¿Qué se puede demostrar en la sustentación?

- **Tres nodos reales** descubriéndose automáticamente.
- **Envío de datos y tareas** desde un nodo hacia otro, y cómo el nodo receptor usa su modelo de IA para procesarlos.
- **Panel de monitoreo** en el navegador mostrando la tabla de vecinos y métricas en vivo.
- **Apagado de un nodo** durante la ejecución de una tarea y **reasignación automática** (el modelo de IA se sigue ejecutando en otro nodo).
- **Métricas** de latencia, tiempo de asignación, tiempo de ejecución y tasa de fallos.

11.7 Conclusión de la fase

Con esta fase se cumple el objetivo de **construir el núcleo funcional de ComplexNet API**: una red de al menos tres nodos que se descubren solos, comparten sus recursos, reparten tareas de forma inteligente, **ejecutan modelos de inteligencia artificial de manera distribuida**, detectan fallos y se recuperan sin intervención humana. Sobre esta base se construirán, en la Fase 3, las tres aplicaciones descentralizadas exigidas por el curso.